

Atividade Final da Disciplina de Algoritmos

Analise Estrutural de Proteinas e Deteccao de Pockets

Programas 01 a 06 + Analise com DoGSite3

Relatorio gerado automaticamente a partir dos resultados dos programas.

1. Introducao

Este trabalho consiste em um conjunto de seis programas em Python para o processamento e analise de estruturas de proteínas no formato PDB. Foram utilizados 10 arquivos PDB da pasta **pdbproteins**, em sua maioria variantes de hemoglobina humana. A seguir, descreve-se o problema tratado por cada programa.

Programa 01 - Separacao de Cadeias

Localiza os arquivos PDB, identifica as cadeias polipeptidicas de cada proteina (coluna 22 do formato PDB) e salva cada cadeia em um arquivo separado no padrao X_nomeOriginal.pdb, gerando um relatorio na tela.

Programa 02 - Modularizacao

Reescreve o Programa 01 isolando a logica de separacao em um modulo independente (separador.py). O programa principal apenas localiza os arquivos, chama a funcao do modulo e exibe os resultados.

Programa 03 - Estatisticas Estruturais

Para cada cadeia gerada, calcula o numero de atomos, de residuos e de aminoacidos diferentes, e identifica as cadeias com maior e menor numero de residuos.

Programa 04 - Frequencia de Aminoacidos

Conta a frequencia dos 20 aminoacidos em cada cadeia, calcula a porcentagem de cada um e gera um grafico de barras salvo na pasta graphs.

Programa 05 - Busca de Motivos Estruturais

Procura um padrao de aminoacidos informado pelo usuario (ex.: GLY-LYS-SER) e informa em quais proteínas e cadeias o padrao ocorre, com a posicao.

Programa 06 - Deteccao de Pockets com DoGSite3

Executa automaticamente o software DoGSite3 para cada proteina, organiza os resultados em subpastas, extrai os descritores dos pockets para um arquivo CSV, responde perguntas de analise e gera graficos.

2. Algoritmo (pseudocodigo)

Programa 01 / 02

```
INICIO
  listar arquivos .pdb da pasta pdbproteins
  imprimir nome, caminho e total de arquivos
  criar pasta chainsproteins
  para cada arquivo PDB:
    ler todas as linhas
    para cada linha ATOM/HETATM/TER:
      ler identificador da cadeia (coluna 22)
      agrupar a linha na cadeia correspondente
    para cada cadeia encontrada:
      gravar arquivo X_nomeOriginal.pdb
  imprimir relatorio (proteina, n.cadeias, arquivos)
FIM
```

Programa 03

```
INICIO
  para cada arquivo em chainsproteins:
    atomos <- 0 ; residuos <- conjunto ; aminoacidos <- conjunto
    para cada linha ATOM:
      atomos <- atomos + 1
      adicionar numero do residuo ao conjunto residuos
      se nome do residuo e aminoacido padrao:
        adicionar ao conjunto aminoacidos
    imprimir atomos, |residuos|, |aminoacidos|
  identificar cadeia com maior e menor n. de residuos
FIM
```

Programa 04

```
INICIO
  para cada arquivo em chainsproteins:
    contar residuos de cada um dos 20 aminoacidos
    calcular porcentagem = 100 * contagem / total
    construir grafico de barras e salvar em graphs/
FIM
```

Programa 05

```
INICIO
  ler motivo do usuario e separar por '-'
  para cada arquivo em chainsproteins:
    montar sequencia de residuos na ordem
    procurar o motivo como subsequencia consecutiva
    para cada ocorrencia: imprimir proteina, cadeia, posicao
FIM
```

Programa 06

```
INICIO
  listar arquivos .pdb e criar pasta resultados_dogsite
  para cada proteina:
    criar subpasta e executar dogsite3 (-p -o -d --writeSiteResiduesPDB)
  imprimir relatorio (total, tempo, nomes, erros)
  ler arquivos *_desc.txt e montar resumo_geral_pockets.csv
  responder perguntas de analise e gerar graficos
FIM
```

3. Estrutura do Programa (funcoes)

separador.py

- identificar_cadeias(linhas): agrupa as linhas do PDB por cadeia.
- separar_cadeias(caminho, pasta_saida): grava um arquivo por cadeia.

programa01.py / programa02.py

- listar_arquivos_pdb(pasta): retorna os arquivos .pdb da pasta.
- separar_cadeias_arquivo(nome): separa as cadeias de um arquivo (P01).
- main(): orquestra a localizacao, separacao e o relatorio.

programa03.py

- analisar_cadeia(caminho): retorna (atomos, residuos, aa diferentes).
- main(): monta a tabela e identifica maior/menor cadeia.

programa04.py

- contar_aminoacidos(caminho): conta os residuos de cada aminoacido.
- calcular_porcentagens(contagem): converte contagens em porcentagens.
- gerar_grafico(nome, porcentagens): salva o grafico de barras.

programa05.py

- ler_sequencia_cadeia(caminho): monta a sequencia de residuos.
- buscar_motivo(sequencia, motivo): localiza o padrao consecutivo.
- identificar_proteina_cadeia(nome): extrai proteina e cadeia do nome.

programa06.py

- listar_pdbs(): lista os PDBs de entrada.
- executar_dogsite(nome): executa o DoGSite3 para uma proteina.
- extrair_descritores(): le os *_desc.txt e monta o DataFrame.
- analisar_resultados(df): responde as perguntas de analise.
- gerar_graficos(df): gera os graficos de volume e profundidade.

4. Resultados

4.1 Estatísticas estruturais por cadeia (Programa 03)

Arquivo	Atomos	Resíduos	Aa diferentes
A_pdb1a00	1069	141	19
A_pdb1a01	1069	141	19
A_pdb1a0l	1929	244	20
A_pdb1a0n	5175	14	9
A_pdb1a0u	1069	141	19
A_pdb1a0z	1069	141	19
A_pdb1a1a	1048	106	19
A_pdb1a1b	1020	105	19
A_pdb1a1c	1041	105	19
A_pdb1a1e	1030	104	19
B_pdb1a00	1122	146	19
B_pdb1a01	1115	146	19
B_pdb1a0l	1929	244	20
B_pdb1a0n	22525	58	18
B_pdb1a0u	1124	146	19
B_pdb1a0z	1124	146	19
B_pdb1a1a	1016	102	19
B_pdb1a1b	998	104	19
B_pdb1a1c	995	104	19
B_pdb1a1e	1003	104	19
C_pdb1a00	1069	141	19
C_pdb1a01	1069	141	19
C_pdb1a0l	1929	244	20
C_pdb1a0u	1069	141	19
C_pdb1a0z	1069	141	19
C_pdb1a1a	10	1	1
C_pdb1a1b	10	1	1
C_pdb1a1c	10	1	1
C_pdb1a1e	10	1	1
D_pdb1a00	1122	146	19
D_pdb1a01	1115	146	19
D_pdb1a0l	1929	244	20
D_pdb1a0u	1124	146	19
D_pdb1a0z	1124	146	19
D_pdb1a1a	10	1	1
D_pdb1a1b	10	1	1
D_pdb1a1c	10	1	1
D_pdb1a1e	10	1	1

Tabela 1. Número de átomos, resíduos e aminoácidos diferentes por cadeia. Cadeia com mais resíduos: A_pdb1a0l (244); com menos: cadeias de 1 resíduo (grupos heme/ligantes).

4.2 Frequencia de aminoacidos (Programa 04)

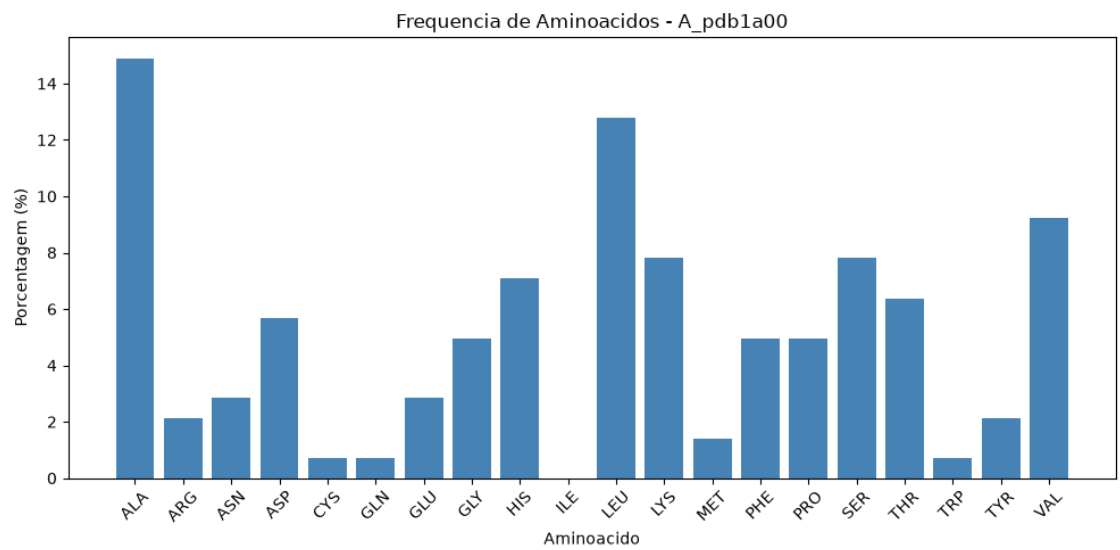


Figura 1. Exemplo de distribuicao de aminoacidos para a cadeia A_pdb1a00 (um grafico e gerado para cada cadeia).

4.3 Deteccao de pockets com DoGSite3 (Programa 06)

Proteina	Pocket	Volume	Superficie	Profund.	Hidrofob.	Acept.	Doad.
pdb1a00	P_1	286.2	421.4	12.87	0.830	5	4
pdb1a01	P_1	280.6	390.6	12.45	0.816	5	4
pdb1a0l	P_1	309.8	475.0	19.86	0.618	14	11
pdb1a0n	P_1	74.8	268.0	5.88	0.611	5	3
pdb1a0u	P_1	284.7	425.7	12.42	0.827	6	3
pdb1a0z	P_1	290.3	371.4	12.42	0.837	5	3
pdb1a1a	P_1	230.9	538.3	10.52	0.562	12	9
pdb1a1b	P_1	119.8	260.2	8.94	0.618	6	9
pdb1a1c	P_1	134.1	369.6	9.90	0.622	12	2
pdb1a1e	P_1	128.5	364.0	9.90	0.639	11	2

Tabela 2. Principal pocket (maior volume) de cada proteina, com os descritores extraidos do DoGSite3. Foram detectados 84 pockets em 10 proteinas.

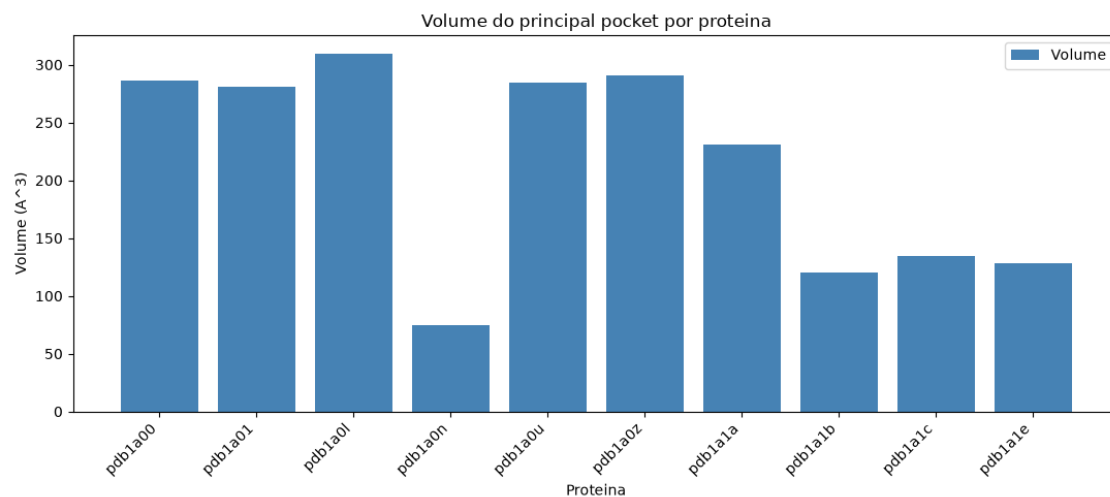


Figura 2. Volume do principal pocket por proteina.

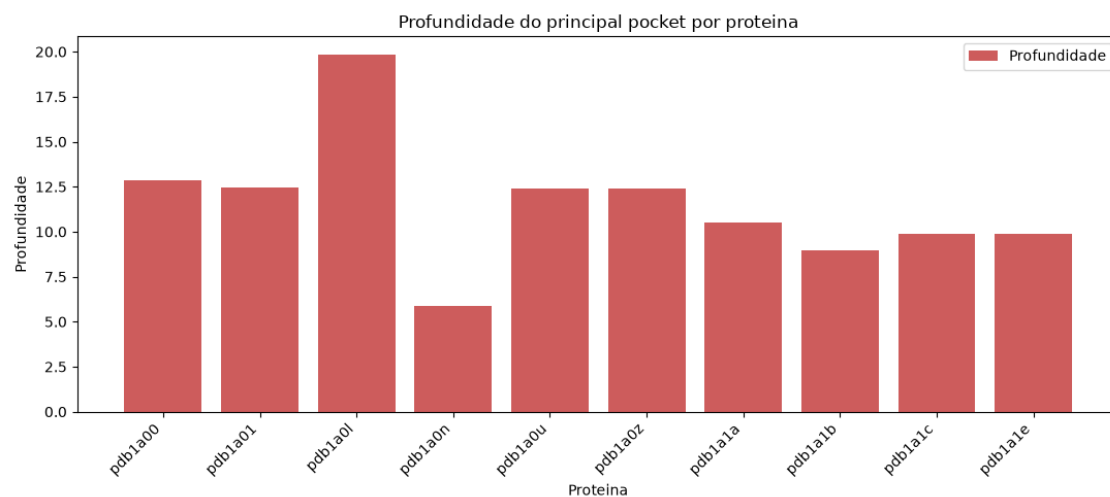


Figura 3. Profundidade do principal pocket por proteina.

4.4 Analise dos resultados

- 1. Maior volume:** pdb1a0l (pocket P_1, volume = 309.8).
- 2. Mais profundo:** pdb1a0l (pocket P_1, profundidade = 19.86).

3. Relacao volume x profundidade: correlacao de Pearson = 0.954, indicando forte relacao positiva (pockets maiores tendem a ser mais profundos).

4. Pockets mais hidrofobicos: pdb1a0u (0.875), pdb1a0u (0.848), pdb1a01 (0.842).

5. Pocket sugerido para docking: pdb1a0l (pocket P_1). Justificativa: apresenta o maior volume (309.8), capaz de acomodar ligantes maiores, alem de boa profundidade (19.86), favorecendo a complementaridade com o ligante.

5. Uso de IA

Qual IA foi utilizada: Claude Code (Anthropic), modelo Claude Opus, como assistente de programação no terminal.

Prompts empregados (resumo):

- Analisar o PDF da atividade final e planejar as entregas.
- Implementar os Programas 01 a 06 em Python conforme os requisitos.
- Usar o comando `dogsit3 -p ... -o ... -d --writeSiteResiduesPDB` no Programa 06.
- Gerar o relatório final em PDF com tabelas, gráficos e seção de uso de IA.

Modificações realizadas manualmente: revisar e ajustar os parâmetros de parsing do formato PDB (colunas de cadeia, resíduo e aminoácido), validar os resultados dos programas com os dados reais, conferir o formato do arquivo de descritores do DoGSite3 e ajustar a seleção de colunas do CSV. (Preencha aqui as edições adicionais que o grupo realizou.)

6. Referências

Volkamer A., Kuhn D., Rippmann F., Rarey M. DoGSiteScorer: A web server for automatic binding site prediction, analysis and druggability assessment. Bioinformatics, 2012.

DoGSite3 - ZBH, Universidade de Hamburgo:
<https://www.zbh.uni-hamburg.de/en/forschung/amd/software/dogsit3.html>